

2024년 2학기 데이터베이스시스템 기말 프로젝트	개발 완료 보고서	문서번호	3
		학번	192195
		작성자	박경준
		작성일자	2024.12.7

기술 블로그 모음 서비스 DB 설계

1. 개념적 모델링(세부 내용)

E-1) COMPANY 개체

· 설명 : 기업 정보를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
company_id	식별자(기본키)	중복 불가, 자동 증가 (AUTO_INCREMENT)	식별자이므로 고유한 값 존재
compnay_name	회사명	중복 불가	고유한 회사 이름
country	기업이 위치한 국가	중복 가능	동일한 국가에 여러 회사 존재 가능
website_url	기업 웹사이트 URL	중복 가능	기업의 계열사 고려
logo_url	기업 로고 이미지 URL (AWS S3 업로드 고려)	중복 가능	기업의 계열사 고려
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

E-2) BLOG 개체

· 설명 : 블로그 글 정보를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
blog_id	식별자(기본키)	중복 불가, 자동 증가 (AUTO_INCREMENT)	식별자이므로 고유한 값 존재
company_id	기업 식별자	중복 가능	하나의 기업에 여러 블로그 글 존재 가능

title	블로그 글 제목	중복 가능	타 기업과의 동일한 제목을 고려한 중복 허용
content	블로그 글 내용	중복 가능	블로그 내용은 고유성이 필요하지 않으므로 중복 허용
blog_url	원문 URL	중복 불가	고유한 원문 주소
published_date	게시 일자	중복 가능	게시 일자는 중복 가능
is_foreign	번역 여부	중복 가능	boolean 타입으로 중복 가능
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

E-3) TRANSLATION 개체

• 설명 : 블로그 글의 번역 데이터를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
blog_id	식별자(기본키)	중복 불가	식별자이므로 고유한 값 존재, BLOG 테이블의 blog_id에 영향을 받음
translated_title	번역된 제목	중복 가능	타 기업과의 동일한 제목을 고려한 중복 허용
translated_content	번역된 내용	중복 가능	블로그 내용은 고유성이 필요하지 않으므로 중복 허용
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

E-4) IMAGE 개체

• 설명 : 블로그 글의 이미지 데이터를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
image_id	식별자(기본키)	중복 불가, 자동 증가 (AUTO_INCREMENT)	식별자이므로 고유한 값 존재
blog_id	블로그 글 식별자	중복 가능	하나의 블로그 글에 여러 이미지 존재 가능
image_url	블로그 글 이미지 URL (AWS S3 업로드 고려)	중복 불가	제목과 달리 내용은 동일할 수 없기 때문에 중복 불가 적용
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

E-5) TAG 개체

• 설명 : 태그 정보를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
tag_id	식별자(기본키)	중복 불가, 자동 증가 (AUTO_INCREMENT)	식별자이므로 고유한 값 존재
tag_type	태그 유형	중복 가능	동일한 유형의 태그를 여러 블로그에서 사용 가능
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

추가 내용) BLOG_TAG 개체

· 설명 : 블로그와 태그 간의 관계를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
blog_tag_id	식별자(기본키)	중복 불가, 자동 증가 (AUTO_INCREMENT)	식별자이므로 고유한 값 존재
blog_id	블로그 글 식별자	중복 가능	다대다 관계(M:N) 표현하기 위한 중복 가능
tag_id	태그 식별자	중복 가능	다대다 관계(M:N) 표현하기 위한 중복 가능
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

E-6) USER 개체

· 설명 : 사용자 정보를 저장하는 테이블

속성명	역할	중복 여부 혹은 특징	이유
user_id	식별자(기본키)	중복 불가, 자동 증가 (AUTO_INCREMENT)	식별자이므로 고유한 값 존재
user_name	사용자명	중복 불가	이메일 내용에 포함 시키기 위한 고유한 사용자명
email	사용자의 이메일	중복 불가	최신 글 알림을 위한 고유한 이메일
is_agreed	동의 여부	중복 가능	boolean 타입을 이용한 여부 파악
is_subscribed	알림 여부	중복 가능	boolean 타입을 이용한 여부 파악
created_at	생성일자	중복 가능	동일한 시간에 생성 가능
updated_at	수정일자	중복 가능	동일한 시간에 수정 가능

2. 논리적 모델링(관계 데이터 모델로의 매핑)

R-1	- COMPANY(<u>company_id</u>, company_name, country, website_url, logo_url, created_at, updated_at) - BLOG(<u>blog_id</u>, company_id(FK), title, content, blog_url, published_date, is_foreign, created_at, updated_at) - company_id(FK): COMPANY(company_id)를 참조하는 외래키로 R-1번 요구사항의 업로드(COMPANY : BLOG = 1 : N)관계를 표현하기 위해서 추가되었다. 즉, 회사는 여러 블로그를 가질 수 있으며, 블로그는 반드시 특정 회사에 속해야 한다. “ON DELETE RESTRICT”를 적용하여 기업 데이터가 삭제 시 연결된 블로그 글을 보호한다.
R-2	- BLOG(<u>blog_id</u>, company_id, title, content, blog_url, published_date, is_foreign, created_at, updated_at) - TRANSLATION(<u>blog_id(FK)</u>, translated_title, translated_content, created_at, updated_at) - blog_id(FK): BLOG(blog_id)를 참조하는 기본키 겸 외래키로 R-2번 요구사항의 수행(BLOG : TRANSLATION = 1 : 1)관계를 표현하기 위해서 추가되었다. 즉, 각 블로그 글은 하나의 번역 데이터를 가질 수도 있으며, 블로그 글 없이 번역은 존재할 수 없다. “ON DELETE CASCADE”를 적용하여 블로그 데이터가 삭제 시 연결된 번역 데이터를 삭제한다.
R-3	- BLOG(<u>blog_id</u>, company_id, title, content, blog_url, published_date, is_foreign, created_at, updated_at) - IMAGE(<u>image_id</u>, blog_id(FK), image_url, created_at, updated_at) - blog_id(FK): BLOG(blog_id)를 참조하는 기본키 겸 외래키로 R-3번 요구사항의 존재(BLOG : IMAGE = 1 : N)관계를 표현하기 위해서 추가되었다. 즉, 각 블로그 글은 여러 이미지를 가질 수 있으며, 이미지는 특정 블로그에 속해야 한다. “ON DELETE CASCADE”를 적용하여 블로그 데이터가 삭제 시 연결된 번역 데이터를 삭제한다.
R-4	- BLOG(<u>blog_id</u>, company_id, title, content, blog_url, published_date, is_foreign, created_at, updated_at) - TAG(<u>tag_id</u>, tag_type, created_at, updated_at) - BLOG_TAG(<u>blog_tag_id</u>, blog_id(FK), tag_id(FK), created_at, updated_at) - 기존 다대다 관계(BLOG : TAG = M : N)을 다중 값 필드가 생기거나 테이블에 대량의 중복 데이터가 포함에 대한 이유로 각각 일대다 관계

	(BLOG : BLOG_TAG = 1 : N, TAG : BLOG_TAG = 1 : N)로 분리한다. ▪ “ON DELETE CASCADE”를 적용하여 블로그 데이터가 삭제 시 연결된 번역 데이터를 삭제한다.
--	---

3. 종속성 판단 및 정규화 과정

E-1 COMPANY	COMPANY(<u>company_id</u> , company_name, country, website_url, logo_url, created_at, updated_at)
	▪ company_id는 기본키로 모든 속성 결정 가능 company_id → company_name, country, website_url, logo_url, created_at, pdated_at ▪ 고유 속성을 통한 종속성 company_name → company_id, country, webiste_url, logo_url, created_at, updated-at

E-2 BLOG	BLOG(<u>blog_id</u> , company_id(FK), title, content, blog_url, published_date, is_foreign, created_at, updated_at)
	▪ blog_id는 기본키로 모든 속성 결정 가능 blog_id → company_id, title, content, blog_url, published_date, is_foreign, created_at, updated_at ▪ 고유 속성을 통한 종속성 blog_url → blog_id, company_id, title, content, published_date, is_foreign, created_at, updated_at ▪ 외래키를 통한 종속성 company_id

E-3 TRANSLATION	TRANSLATION(<u>blog_id(FK)</u> , translated_title, translated_content, created_at, updated_at)
	▪ blog_id는 기본키겸 외래키로 모든 속성 결정 가능 blog_id → translated_title, translated_content, created_at, updated_at ▪ 외래키를 통한 종속성 blog_id

E-4 IMAGE	IMAGE(<u>image_id</u> , blog_id(FK), image_url, created_at, updated_at)
	- image_id는 기본키로 모든 속성 결정 가능 image_id → blog_id, image_url, created_at, updated_at - 고유 속성을 통한 종속성 image_url → image_id, blog_id, created_at, updated_at - 외래키를 통한 종속성 blog_id

E-5 TAG	TAG(<u>tag_id</u> , tag_type, created_at, updated_at)
	tag_id는 기본키로 모든 속성 결정 가능 tag_id → tag_type, created_at, updated_at

E-6 BLOG_TAG	BLOG_TAG(<u>blog_tag_id</u> , blog_id(FK), tag_id(FK), created_at, updated_at)
	blog_tag_id는 기본키로 모든 속성 결정 가능 blog_tag_id → blog_id, tag_id, created_at, updated_at (blog_id, tag_id) → created_at, updated_at

E-7 USER	USER(<u>user_id</u> , user_name, email, is_agreed, is_subscribed, created_at, updated_at)
	- user_id는 기본키로 모든 속성 결정 가능 user_id → user_name, email, is_agreed, is_subscribed, created_at, updated_at - 고유 속성을 통한 종속성 email → user_name, is_agreed, is_subscribed, created_at, updated_at user_name → user_id, email, is_agreed, is_subscribed, created_at, updated_at

4. SQL 코드 구현 및 무결성 판단

4- 1. 데이터베이스 생성	
소스코드	<pre>-- 데이터베이스 생성 CREATE DATABASE 박경준; -- 데이터베이스 선택 USE 박경준;</pre>
4- 2. 테이블 생성	
소스코드	<pre>-- 기업 테이블 생성 CREATE TABLE COMPANY (COMPANY_ID BIGINT(20) NOT NULL AUTO_INCREMENT, COMPANY_NAME VARCHAR(255) UNIQUE NOT NULL, COUNTRY VARCHAR(255) NOT NULL, WEBSITE_URL VARCHAR(255) NOT NULL, LOGO_URL VARCHAR(255) NOT NULL, CREATED_AT DATETIME(6) NOT NULL, UPDATED_AT DATETIME(6) NOT NULL, PRIMARY KEY (COMPANY_ID)) ENGINE=InnoDB; -- 블로그(글) 테이블 생성 CREATE TABLE BLOG (BLOG_ID BIGINT(20) NOT NULL AUTO_INCREMENT, COMPANY_ID BIGINT(20) DEFAULT NULL, TITLE VARCHAR(255) NOT NULL, CONTENT TEXT NOT NULL, BLOG_URL VARCHAR(255) UNIQUE NOT NULL, PUBLISHED_DATE DATETIME(6) NOT NULL, IS_FOREIGN TINYINT(1) NOT NULL DEFAULT 0, CREATED_AT DATETIME(6) NOT NULL, UPDATED_AT DATETIME(6) NOT NULL, PRIMARY KEY (BLOG_ID), FOREIGN KEY (COMPANY_ID) REFERENCES COMPANY (COMPANY_ID) ON DELETE RESTRICT) ENGINE=InnoDB; -- 번역 테이블 생성 CREATE TABLE TRANSLATION (</pre>


```

BLOG_ID          BIGINT(20)  NOT NULL,
TRANSLATED_TITLE  VARCHAR(255) NOT NULL,
TRANSLATED_CONTENT TEXT  NOT NULL,
CREATED_AT        DATETIME(6) NOT NULL,
UPDATED_AT        DATETIME(6) NOT NULL,
PRIMARY KEY (BLOG_ID),
FOREIGN KEY (BLOG_ID) REFERENCES BLOG (BLOG_ID)
ON DELETE CASCADE
) ENGINE = InnoDB;

-- 이미지 테이블 생성
CREATE TABLE IMAGE
(
    IMAGE_ID          BIGINT(20)          NOT NULL
    AUTO_INCREMENT,
    BLOG_ID          BIGINT(20)  DEFAULT NULL,
    IMAGE_URL        VARCHAR(255) UNIQUE NOT NULL,
    CREATED_AT        DATETIME(6) NOT NULL,
    UPDATED_AT        DATETIME(6) NOT NULL,
    PRIMARY KEY (IMAGE_ID),
    FOREIGN KEY (BLOG_ID) REFERENCES BLOG (BLOG_ID)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- 태그 테이블 생성
CREATE TABLE TAG
(
    TAG_ID            BIGINT(20)          NOT NULL
    AUTO_INCREMENT,
    TAG_TYPE          VARCHAR(255) NOT NULL,
    CREATED_AT        DATETIME(6) NOT NULL,
    UPDATED_AT        DATETIME(6) NOT NULL,
    PRIMARY KEY (TAG_ID)
) ENGINE = InnoDB;

-- BLOG / TAG 관계 테이블 생성
CREATE TABLE BLOG_TAG
(
    BLOG_TAG_ID        BIGINT(20)          NOT NULL
    AUTO_INCREMENT,
    BLOG_ID            BIGINT(20)  DEFAULT NULL,

```

	<pre> TAG_ID BIGINT(20) DEFAULT NULL, CREATED_AT DATETIME(6) NOT NULL, UPDATED_AT DATETIME(6) NOT NULL, PRIMARY KEY (BLOG_TAG_ID), FOREIGN KEY (BLOG_ID) REFERENCES BLOG (BLOG_ID) ON DELETE CASCADE, FOREIGN KEY (TAG_ID) REFERENCES TAG (TAG_ID) ON DELETE CASCADE) ENGINE=InnoDB; -- 사용자 테이블 생성 CREATE TABLE USER (USER_ID BIGINT(20) NOT NULL AUTO_INCREMENT, USER_NAME VARCHAR(255) UNIQUE NOT NULL, EMAIL VARCHAR(255) UNIQUE NOT NULL, IS_AGREED TINYINT(1) NOT NULL DEFAULT 0, IS_SUBSCRIBED TINYINT(1) NOT NULL DEFAULT 0, CREATED_AT DATETIME(6) NOT NULL, UPDATED_AT DATETIME(6) NOT NULL, PRIMARY KEY (USER_ID)) ENGINE=InnoDB; </pre>
4- 3. Table에 데이터 삽입	
소스코드	<pre> -- COMANY 데이터 삽입 INSERT INTO COMPANY (COMPANY_NAME, COUNTRY, WEBSITE_URL, LOGO_URL, CREATED_AT, UPDATED_AT) VALUES ('카카오', 'Korea', 'https://kakaocorp.com', 'https://kakaocorp.com/logo.png', NOW(), NOW()), ('네이버', 'Korea', 'https://naver.com', 'https://naver.com/logo.png', NOW(), NOW()), ('Meta', 'USA', 'https://about.meta.com/ko/', 'https://about.meta.com/ko/logo.png', NOW(), NOW()); -- BLOG 데이터 삽입 INSERT INTO BLOG (COMPANY_ID, TITLE, CONTENT, BLOG_URL, PUBLISHED_DATE, IS_FOREIGN, CREATED_AT, UPDATED_AT) VALUES (1, 'API 설계란?', '본문은 어떻게 하면 API를 잘 통합하고 설계 할 수 있을지에 대한 내용을 설명합니다.', 'https://kakaocorp.com/api-guide', '2024-11-11 12:00:00', 0, NOW(), NOW()), (2, '좋은 데이터베이스 설계란?', '본문은 어떻게 하면 데이터베 </pre>

	이스 설계를 잘 할 수 있을지에 대한 내용을 설명합니다.', 'https://naver.com/data-architect', '2024-11-15 09:30:00', 0, NOW(), NOW()), (3, 'Advancements in AI', 'Exploring the latest trends in AI research.', 'https://about.meta.com/ko/ai-trends', '2024-11-22 14:00:00', 1, NOW(), NOW()); -- TRANSLATION 데이터 삽입 INSERT INTO TRANSLATION (BLOG_ID, TRANSLATED_TITLE, TRANSLATED_CONTENT, CREATED_AT, UPDATED_AT) VALUES (3, 'AI 발전', 'AI 연구의 최신 동향을 탐구합니다.', NOW(), NOW()); -- IMAGE 데이터 삽입 INSERT INTO IMAGE (BLOG_ID, IMAGE_URL, CREATED_AT, UPDATED_AT) VALUES (1, 'https://kakaocorp.com/images/api-guide.jpg', NOW(), NOW()), (2, 'https://naver.com/images/data-architect.jpg', NOW(), NOW()), (3, 'https://about.meta.com/ko/images/ai-trends.jpg', NOW(), NOW()); -- TAG 데이터 삽입 INSERT INTO TAG (TAG_TYPE, CREATED_AT, UPDATED_AT) VALUES ('API', NOW(), NOW()), ('BE', NOW(), NOW()), ('AI', NOW(), NOW()); -- BLOG_TAG 데이터 삽입 INSERT INTO BLOG_TAG (BLOG_ID, TAG_ID, CREATED_AT, UPDATED_AT) VALUES (1, 1, NOW(), NOW()), (2, 2, NOW(), NOW()), (3, 3, NOW(), NOW()); -- USER 데이터 삽입 INSERT INTO USER (USER_NAME, EMAIL, IS_AGREED, IS_SUBSCRIBED, CREATED_AT, UPDATED_AT) VALUES ('박경준', 'kyeongjun@naver.com', 1, 1, NOW(), NOW()),
--	--

	('이도연', 'doyeon@gmail.com', 1, 0, NOW(), NOW()), ('James', 'james@gamil.com', 0, 1, NOW(), NOW());
--	---

5. 데이터베이스 검증

5- 1. 각 Table의 데이터 검색

소스코드

-- COMPANY 테이블 확인

SELECT * FROM COMPANY;

-- BLOG 테이블 확인

SELECT * FROM BLOG;

-- TRANSLATION 테이블 확인

SELECT * FROM TRANSLATION;

-- IMAGE 테이블 확인

SELECT * FROM IMAGE;

-- TAG 테이블 확인

SELECT * FROM TAG;

-- BLOG_TAG 테이블 확인

SELECT * FROM BLOG_TAG;

-- USER 테이블 확인

SELECT * FROM USER;

결과

-- COMPANY

COMPANY_ID	COMPANY_NAME	COUNTRY	WEBSITE_URL	LOGO_URL	CREATED_AT	UPDATED_AT
1	카카오	Korea	https://kakaocorp.com	https://techcorp.com/logo.png	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
2	네이버	Korea	https://naver.com	https://datasolutions.co.kr/logo.png	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
3	Meta	USA	https://about.meta.com/ko/	https://https://about.meta.com/ko/logo.png	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

-- BLOG

BLOG_ID	COMPANY_ID	TITLE	CONTENT	BLOG_URL	PUBLISHED_DATE	IS_FOR...	CREATED_AT	UPDATED_AT
1	1	1 API 소개?	본문은 이렇기 위한 API를 잘 설명하고	https://kakaocorp.com/api-	2024-11-11 12:00:00.0	0	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
2	2	2 최신 AI/ML 트렌드	본문은 이렇기 위한 데이터베이스 설계	https://naver.com/data-arc-	2024-11-15 09:30:00.0	0	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
3	3	3 Advancements in AI	Exploring the latest trends in	https://about.meta.com/ai-	2024-11-22 14:00:00.0	1	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

-- TRANSLATION

BLOG_ID	TRANSLATED_TITLE	TRANSLATED_CONTENT	CREATED_AT	UPDATED_AT
3	AI 발전	AI 연구의 최신 동향을 탐구합니다.	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

-- IMAGE

IMAGE_ID	BLOG_ID	IMAGE_URL	CREATED_AT	UPDATED_AT
1	1	https://kakaocorp.com/images/api-guide.jpg	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
2	2	https://naver.com/images/data-architect.jpg	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
3	3	https://about.meta.com/ko/images/ai-trends.jpg	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

-- TAG

	TAG_ID	TAG_TYPE	CREATED_AT	UPDATED_AT
1	1	API	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
2	2	BE	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
3	3	AI	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

-- BLOG_TAG

	BLOG_TAG_ID	BLOG_ID	TAG_ID	CREATED_AT	UPDATED_AT
1	1	1	1	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
2	2	2	2	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
3	3	3	3	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

-- USER

	USER_ID	USER_NAME	EMAIL	IS_AGREED	IS_SUBSCRIBED	CREATED_AT	UPDATED_AT
1	1	박영준	kyeongjun@naver.com	1	1	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
2	2	이도현	doyeon@gmail.com	1	0	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000
3	3	James	james@gmail.com	0	1	2024-12-07 20:40:46.000000	2024-12-07 20:40:46.000000

6. 데이터베이스 활용 예시

6- 1. 회사별 가장 최근 작성된 블로그 조회

소스코드

SELECT C.COMPANY_NAME, B.TITLE, B.PUBLISHED_DATE AS 최신게시물_업로드일자
FROM COMPANY C
JOIN BLOG B ON C.COMPANY_ID = B.COMPANY_ID
WHERE B.PUBLISHED_DATE = (
SELECT MAX(B2.PUBLISHED_DATE)
FROM BLOG B2
WHERE B2.COMPANY_ID = C.COMPANY_ID
);

결과

	COMPANY_NAME	TITLE	최신게시물_업로드일자
1	Meta	Advancements in AI	2024-11-22 14:00:00.000000
2	네이버	좋은 데이터베이스 설계란?	2024-11-15 09:30:00.000000
3	카카오	API 설계란?	2024-11-11 12:00:00.000000

6- 2. 특정 태그가 포함된 블로그 글 조회					
소스코드	<pre>SELECT TITLE FROM BLOG WHERE BLOG_ID IN (SELECT BT.BLOG_ID FROM BLOG_TAG BT WHERE BT.TAG_ID IN (SELECT TAG_ID FROM TAG WHERE TAG_TYPE = 'AI'));</pre>				
결과	<table><tr><th></th><th>TITLE</th></tr><tr><td>1</td><td>Advancements in AI</td></tr></table>		TITLE	1	Advancements in AI
	TITLE				
1	Advancements in AI				

6- 3. 번역된 블로그의 회사별 개수

소스코드

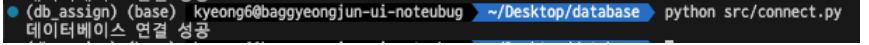
SELECT C.COMPANY_NAME, COUNT(T.BLOG_ID) AS 번역된_블로그개수
FROM COMPANY C
LEFT JOIN BLOG B ON C.COMPANY_ID = B.COMPANY_ID
LEFT JOIN TRANSLATION T ON B.BLOG_ID = T.BLOG_ID
GROUP BY C.COMPANY_NAME;

결과

	COMPANY_NAME	번역된_블로그개수
1	Meta	1
2	네이버	0
3	카카오	0

7. 응용프로그램(Python)과 MySQL 연동 및 활용 예시 수행

· 코드에 대한 자세한 내용은 주석 및 README.pdf에 작성

7- 1. 작성한 스크립트	
역할	1. connect.py 해당 스크립트는 데이터베이스(MySQL)와 연결을 하기 위한 스크립트이다. 2. models.py 해당 스크립트는 사전에 정의한(sql문) 스키마로 데이터베이스의 구조와 일치시키는 스크립트이다. 3. crud.py 해당 스크립트는 “6.데이터베이스 활용 예시”의 쿼리문을 python을 이용하여 수행하는 로직이 존재하는 스크립트이다.
결과	- 데이터베이스 연결 확인(명령어 : python src/connect.py) 위 명령어 중 python 이후 명령어는 현재 경로를 기준으로 connect.py가 존재하는 경로를 작성하면 된다.  <pre>(db_assign) (base) kyeong6@baggyeongjun-ui-notebug ~/Desktop/database python src/connect.py 데이터베이스 연결 성공</pre> - 활용 예시(명령어 : python src/crud.py) 위 명령어 중 python 이후 명령어는 현재 경로를 기준으로 crud.py가 존재하는 경로를 작성하면 된다.  <pre>(db_assign) (base) kyeong6@baggyeongjun-ui-notebug ~/Desktop/database python src/crud.py 회사별 가장 최근 작성된 블로그 조회: [{'COMPANY_NAME': 'Meta', 'TITLE': 'Advancements in AI', '최신게시물_업로드일자': datetime.datetime(2024, 11, 22, 14, 0)}, {'COMPANY_NAME': '네이버', 'TITLE': '좋은 데이터베이스 설계란?', '최신게시물_업로드일자': datetime.datetime(2024, 11, 15, 9, 30)}, {'COMPANY_NAME': '카카오', 'TITLE': 'API 설계란?', '최신게시물_업로드일자': datetime.datetime(2024, 11, 11, 12, 0)}] 'MI' 태그가 포함된 블로그 조회: [{'TITLE': 'Advancements in AI'}] 번역된 블로그의 회사별 개수 조회: [{'COMPANY_NAME': 'Meta', '번역된_블로그개수': 1}, {'COMPANY_NAME': '네이버', '번역된_블로그개수': 0}, {'COMPANY_NAME': '카카오', '번역된_블로그개수': 0}]</pre>